

SF 2025 AORISTOS



分冊③

目次

1. 仮想通貨と電子決済の仕組み(根来)
2. 日常へ IT 設計という概念を持ち込んでみる(根来)
3. 定理証明支援系(根来)
4. 円周率の計算(根来)
5. 現代の LLM について(根来)
6. 僕の考えた最強の Web3(根来)

この分冊③では、応用的な科学技術特に情報についてのトリビアを紹介します。

基礎研究サイドの方にはあまりおすすめしません。

仮想通貨と電子決済の仕組み

仮想通貨への投資を勧める記事では決してありません

まずは分散型システムと暗号化技術の代表として仮想通貨やその周辺の分散型システムの仕組みについて説明していきます。仮想通貨に管理にはブロックチェーンという技術が使われています。そのブロックチェーンについて説明してきます。

そもそも、仮想通貨を実現するにはどうすればいいかというと、

「人がその通貨をやり取りしたことを正確に記録する信頼できる台帳」を作ればいいです。そうしたらそれぞれの人の持っている通貨の量を一意的かつ信頼を持って言うことができます。

しかし台帳を誰か特定の人(例えばサービス管理運営者が持っている)、その運営者を信頼しないといけなくなって仮想通貨システムとして成立しないです。よってサービスを利用しているユーザー全員が共同で台帳を管理していちいち同期するようにすればいいというわけです。

ここで前提知識としてハッシュ関数を説明します。ハッシュ関数とは、様々なデータを入力値として与えると、0と1の数の並びが出力される関数です。

性質にはこんなものがあります。

- 1、入力値を少しでも変えると全く違うものが返ってくる。
- 2、ある01の並びを出力するような入力を答える時に、「いろんな入力をしてみてその並びと等しいものを探す」という方法より効率的な方法をまだ人類が知らない。

イメージは素因数分解です。

例えば、 $37 \times 31 = 1147$ だったら暗算はできなくても筆算を書いたらすぐすることができます。しかし、1147の素因数は？という問いに答えるのは結構大変です。具体的には割る2・割る3・割る5というように小さい素数から割り切れるかを確認していく必要があります。sqrt(1147)までの素数で全て割り算したら絶対に1147がどのように素因数分解されるかを言うことができます。よって入力される大きな素数っぽい数のスケールの二乗根に比例した回数の割り算をしないといけなくなります。しかし逆向きは1回の計算で済みます。

ちなみに素因数分解と今回システムに使っているハッシュ関数(SHA256と言う)は全く関係ないですが、素因数分解を利用した暗号化技術にRSA暗号があり、Webサイトなどにアクセスする時にも実際に使われています。ちなみに量子コンピュータが話題になっているのは、「ある01の並びを出力するような入力を答える」問題を効率的に解ける(あくまでも)可能性があるからです。

今度は公開鍵暗号について説明します。一般的に思いつく暗号化の方法は、秘密鍵暗号と呼ばれるもので次のように行われます。秘密鍵暗号では「平文→暗号」と「暗号→平文」は同じ鍵を使って行います。秘密鍵が漏れると解読されてしまいますから、どうにか最初に秘密鍵を何らかの安全な環境で送り届ける必要があります。これとは対照的に公開鍵暗号では、「平文→暗号」と「暗号→平文」は異なる鍵で行います。「平文→暗号」には公開鍵を、「暗号→平文」には秘密鍵を使います。さっきのハッシュ関数と同じように、暗号化はできるけどその暗号を解読しろといわれてもいろんな平文を暗号化し

てみるという方法よりも効率的な暗号の解き方が存在しないというものです。そうするとそもそもバレたら困る鍵を渡す必要がありませんから安全というわけです。現実的にはこの公開鍵暗号は計算に大変な時間がかかるので、秘密鍵暗号の秘密鍵だけ公開鍵暗号で安全に届けそのあとは秘密鍵暗号で通信するというハイブリッドな方法が使われています。

まずは前提知識としてデジタル署名について解説します。デジタル署名とは「誰かが承認していることを証明するもの」です。デジタル署名はハッシュ関数を使って実現できます。Aさんが承認しているということをBさんが確認するということを考えると、次のようなアルゴリズムでデジタル署名が行われます。

ここでは、

- 署名は鍵があればすぐに生成することができる
- 署名は鍵がなければ生成することは難しい
- 署名は鍵がなくてもそれが正しいことがすぐわかる

ような関数 f を使います。

※一部説明の都合上簡略化しています。

1. Aさんが関数 f と鍵を使って「承認した」という旨のメッセージを署名にし、公開します。
2. 署名をBさんが見て関数 f を使って正しさを証明できます。なぜなら正しい署名を作れる人はAさんだけだからです。

それでは、これからブロックチェーンというアイデアを再発明していきましょう。そのステップを見ることでこのアイデアの素晴らしさがわかってくると思います。

まずは台帳を署名を使って作っていくということを考えてみます。台帳とはここでは仮想通貨の所有者の取引の記録です。具体的には以下のようなものです。

- AさんがBさんに100p渡した
- DさんがCさんに280p渡した

これらをどこかに溜めていきます。そうしたら全ての取引を正しくきちんと把握することができるようになるはずですが、しかしこのメカニズムには致命的な欠陥が複数あります。

- 中央集権的であること。
- 順序がわからないこと。

中央集権的であるとそれ用のサーバーを使っている人が不正をしていないことを証明するのが非常に難しくなります。新しい取引を勝手に作ることはデジタル署名的に不可能だったとしても何らかの取引を隠蔽することは簡単にできてしまいます。またもしこのサーバーが落ちたら仮想通貨は使えなくなってしまうし、多大な運用コストがかかります。

また、順序がわからないと残金が100pなのに、40pと80pの買い物を同時にすることが可能になりま

す。どちらかができないようになるべきですが、それを防ぐ方法がありません。逆にいうと順序を保証できたらこの問題は解決します。

よって仮想通貨を使う人全員で、順序を持った台帳のコピーを持ち合いそれぞれが正しさを検証すれば良さそうです。しかしそれでは誰かが「自分が100pもらう」という取引を勝手に追加したときに、それが広がってしまいます。またほとんど同時に行われた取引の順序を決めることが難しいです。

そこでブロックチェーンを導入します。

ブロックチェーンとは一言で言うなら「台帳を全員で安全に管理する仕組み」です。

AさんがBさんに100p渡した

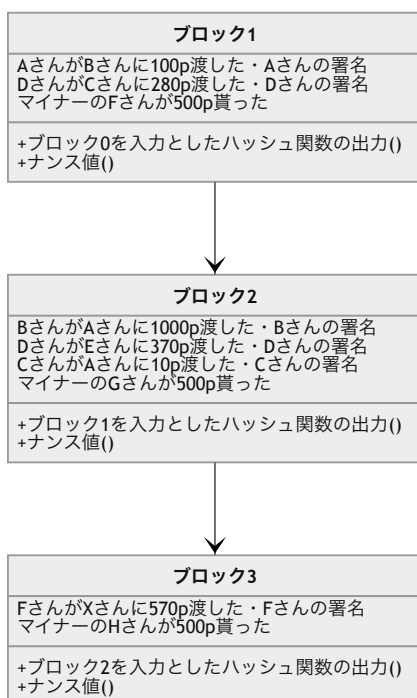
DさんがCさんに280p渡した

このような台帳があれば誰が何p持っているのかわかります。しかしその台帳が正しいことをみんなで保証しようという仕組みが必要です。

ひとまずなぜこのようなルールなのかわかなくてもいいので、全体像を把握しましょう。

台帳をひとまとまりに分けてブロックという単位で管理することにします。ブロックの中には、以下の図のように複数の通貨のやり取りの記録それに対応するデジタル署名と、一個前のブロックを入力とした時のハッシュ関数の出力(ハッシュ)と、ハッシュがたまたま00000から始まるハッシュ関数の入力(ナンス値)が入ることにします。

実はこのブロックの列(ブロックチェーン)は枝分かれすることもあります。クライアント(仮想通貨を使う人)は最も長いブロックチェーンを信用します。

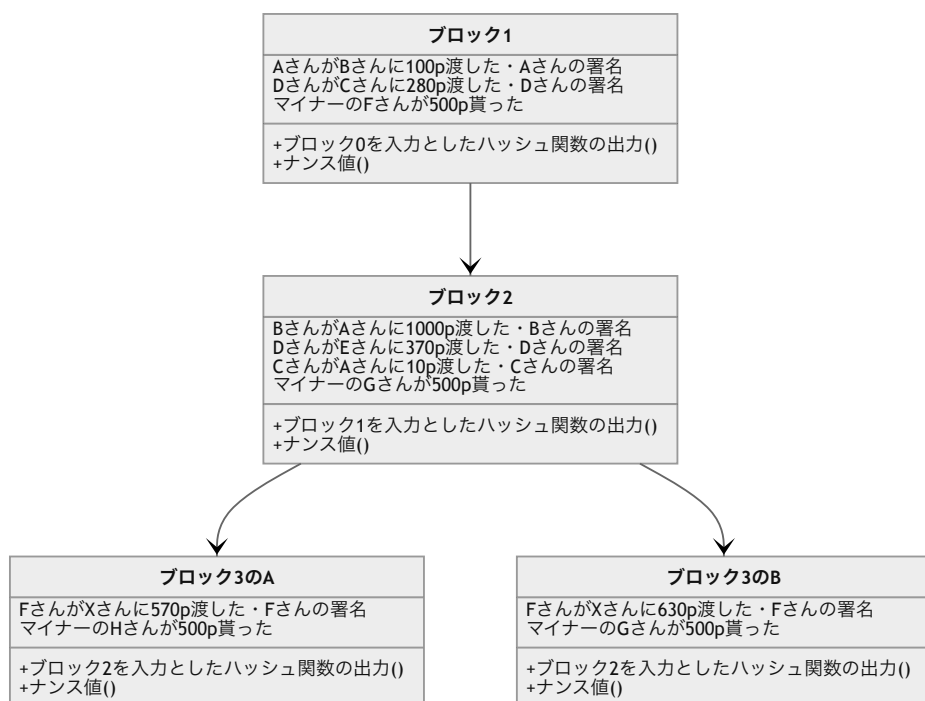


さて、この構造にしたらなぜこの台帳が改竄されていないことがわかるのかを説明していきます。
まずはブロックの k 番目について考えていきます。

k 番目のブロックには、 $k-1$ 番目のブロックのハッシュ値及び取引情報(コンテンツ)が最初に入っています。コンテンツの長さが一定の数値を超えるとナンス値が生成されます。コンピュータ資源を活用して頑張ってたまたま000000から始まるハッシュ値の入力側を探します。この操作はマイニングと呼ばれます。マイニングをしているマイナーは世界にたくさんいて、早い者勝ちで自分の作ったナンス値をブロックの中に入れることができます。ナンス値が見つかったら今度は k 番目のブロックのハッシュ値が取られて取引情報がどんどん書き込まれていきます。その後は同じ手順です。この時マイナーのなかで最初にナンス値を見つけられた人は一定の仮想通貨上での金額を受け取ることができます。なのでみんなマイニングをしようとするわけですね。これは仕組み上必須なものと捉えることもできますが、仮想通貨の発行源と見ることもできます。ちなみにこの時、具体的な情報を何も提供していないのにマイニングをしたことが証明されているのでこれは不思議だということで「プルーフオブワーク」と言われています。

台帳を改竄するには、正しいブロックチェーンとは別の、自分が有利なコンテンツを書き込んだブロックチェーンを作る必要があります。しかし、それは先ほど説明したデジタル署名の仕組みによって不可能になっています。

ではデジタル署名だけでは解決できなかった存在保証と順序性の保証をどう解決するかを解説します。
例えばたまたま同時にブロックが繋がられ、分岐してしまったとします。



そうすると、マイナーたちは確率的にどちらのブロックの後にブロックを繋げるかを決め、いつも通りマイニングをします。するとどちらかが1つだけ伸びます。そうするとその伸びた方が正しいブロックチェーンとマイナーの間でみなされその後に全てのマイナーがブロックを繋げようとしようとします。するとどれか一つのブロックチェーンが正しいものとして同期されます。

ここで間違った取引が含まれているようなブロックは、マイナーがそのブロックの有効性を確認してからマイニングを行うため次にブロックが繋がることはありません。

また正しい取引も最短でその唯一のブロックチェーンに取り込まれないかもしれませんが、チェーンに入っていない取引情報も未認証ながら全体で共有されますから、次のブロックに入ることができるわけです。それが無理ならまたその次のブロックに...

ここで、攻撃者の気持ちでこのシステムをもう一度見てみましょう。攻撃者のしたいことは主に2通りです。

- 勝手に取引を追加する
- 既存の取引の時間的順序の入れ替え・取引の削除

取引を勝手に追加することはデジタル署名の仕組みの上できません。デジタル署名はその有効性の証明は簡単にできるが、有効な署名を新しく作るのは非常に難しいという性質を持っていました。

既存の取引の時間順序の入れ替え・取引の削除は、ブロックチェーン上の理由からできません。もし昔のブロックを入れ替えるとします。そうするとハッシュ値の性質からその次のハッシュ値が変わってしまうので、その次のハッシュ値も変わって...となり、その以後のブロックのナンス値を全て自分で作っていく必要があります。またクライアントは最も長いブロックチェーンが正しいと判断するため、その攻撃者の作ったブロックチェーンを他のブロックチェーンとブロックを繋げる作業の競争をする必要があります。これは、この仮想通貨のマイナーの計算資源の50%以上を確保することが必要でこれはほとんど不可能です。なぜなら攻撃者以外のマイナーは攻撃者の作ったチェーンが正しくないことを確認しそれにブロックを繋げることは全くないからです。

ブロックチェーンを理解できましたか？僕はいつもこのようなよくできたアイデアに触れると、複雑かつ単純というどこか逆説的な感覚に陥ります。わからなかったら間を空けてもう一度読んでみてください。どうしてもわからなければ、僕なんかよりもうまく説明している動画がありますのでぜひご覧ください。

<https://youtu.be/o-xhOBn4S60?si=Ihoy3rWDtMi7qE09>

日常へIT設計という概念を持ち込んでみる

はじめに

日常生活でのIT設計を考えてみました。

※これは実際に当てはめてみたらという視点で設計について説明している文章であり、それが現実世界でも有用であるかどうかは保証しません。

突然ですが、みなさんは「プログラマ」と聞くと何をイメージしますか？「パソコンの黒い画面を開き高速でキーボードを叩いている」という姿を想像する方が多いのではないかと思います。しかし実はそんなことはありません。実際は「設計」という作業にもかなりウェイトをかけていることが多いです。

設計に時間をかけるような開発ステップのことを"Water Fall Model"といい古くからある開発手法の一つで「完璧な設計を作る」という精神のものです。逆に最近流行っているのが真逆の"Ajail Model"で、これは「実際の成果物がないと意味ないよね」という精神の開発手法です。どちらがいいというわけではありませんが、Google・Amazonなど世界的IT企業は"Ajail Model"を採用している傾向にあります。これはIT企業が世の中に増え競争相手が増えたことから、「設計が甘くなったとしてもできるだけ早くユーザーにソフトウェアを届ける」という考え方が増えたからではないかと言われています。

"かなりウェイトをかける"というのは"Water Model"の場合の話であり現代の開発に同じ枠が適用できるかという点に怪しいです。ただ"Ajail Model"も設計を軽んじているわけではありません。

実際僕は今回のSFの公式Webサイトを開発した人の1人ですが、設計の重要性を深く実感することになりました。

プログラミングの設計思想を学ぶことで、私たちの日常の課題解決能力を向上させることができる。かもしれません...

設計とは

設計が大事と主張してきたものの設計とは何かについて説明していなかったですね。IT開発における設計とは、これから行う作業のためにベースライン・方法などをまとめておくことを言います。プログラミングというタスクの具体的な指示書を作るイメージです。具体的に、スーパーマリオシリーズの最高傑作である（と思う）スーパーマリオオデッセイを作っている人の気持ちになってみましょう。

- どんなゲームを作るのか決める キャプチャ（敵に乗り移るアクション）がセールスポイント

ゲームの詳細を決める どんなワールドがあってどんなストーリーがあって敵はこんなのがいてアイテムにはこれがある...

- どんなふうに実装するか考える アイテムを管理するためのプログラムはこんな感じ、マリオを動かすプログラムはこんな感じ、これは具体的にはこんなアルゴリズムをしたら効率的だな...
- どうやってプログラムを整理するか考える アイテムを管理するプログラムはここに置こう、あ、ここに共通のコードがあるからどこからでも使えるようにここに置いておこう...
- プログラムを書く よし実装しよう。設計をきちんとしたおかげで何をすればいいか頭が整理されているなあ...

設計をせずにいきなりプログラムを書き始めると、キャプチャ機能のプログラムがマリオの移動プログラムとごちゃ混ぜになってしまい、後から修正するのが大変になるということもあります。

ゲームデザインの仕事も含まれているので実際にはここまでの作業を同じ人がすることはほとんどないかと思いますが、やっていることは同じようなことです。

きちんと説明しようとする専門用語が飛び出すので少し嘘をついています。

日常で言うと、「今晚はカレーがいい」よりも

「今晚はカレーを作ってください。付属の材料リストを参照し冷蔵庫の中にあるものをリストアップしてください。ネットでスーパーのチラシを閲覧しセール状態であるかどうか確認してください。もし近頃不足する材料が安くなることが判明しその差額が600円以上になるのであれば指示に反抗してカレー以外を作ることも検討してください。その場合その旨を伝えてください。もしカレーを作ることが決定したら夕方の混む時間よりも前にスーパーに行きリストアップしたものを購入してください。ポイントカードを使用すること。購入後帰宅しカレーの調理を始めてください。...」

の方が絶対いいですね。頭を使わなくていいので楽です。

でも逆に言えばこの「設計」を作ることにはかなり時間がかかりそうですね。「もしカレーを作らない時」についてさえ言及しているからです。カレーを作るならば絶対に必要のない思考を設計する時にしてしまいます。なので逆に非効率です。

つまり設計はあくまでもトレードオフで、日常を変えてしまうゲームチェンジャーというわけではありません。

「良いコードと悪いコードで学ぶ設計入門」という僕の愛読書の言葉を引用します。

設計にbestはありません。常にbetterを目指しましょう。

原則

設計の文脈での原則とは、betterな設計を作るための基本的なルールのことです。ここからは少しプログラミングの用語が出てきますが、それぞれ日常の用語に変換することができます。

SOLID原則

"SOLID"というのは5つの原則の頭文字をつなげたものです。

Single Responsibility Principle (単一責任の原則)

「クラスやモジュールはただ一つの責任を持つべきである。」

手順書や部署は一つの責任を持つべきです。カレーの作り方にハンバーグの作り方が書かれていたらそれを使う人は混乱しますし、後でハンバーグの作り方を知りたいときにまさかカレーの作り方の中にあるとは思わないので探すのに苦労します。またハンバーグの作り方がカレーの作り方以外の他の手順書にも分割して書かれていたら、カレーの作り方を変更した際にハンバーグの作り方についても他の手順書と矛盾していないことを確認しなければいけません。

カレーの作り方

「カレーを作るには野菜を切って野菜を炒める必要がありますが、ハンバーグを作るときも玉ねぎを別で炒める必要があります。さて野菜を入れる順番は...」

他の手順書

「シャキシャキ感を残すためにハンバーグは玉ねぎを生でタネと混ぜて焼きます。」

玉ねぎの扱いについて思いっきり矛盾していますが、それに気づくのは別の紙に書かれている内容のため気づきにくいでしょう。

つまり玉ねぎについての扱いを修正するためには他の手順書に書かれているかもしれない玉ねぎの扱いも同時に確認して同時に修正する必要があります。これはこの「カレーの作り方」という手順書を更新する係の人がカレー以外の手順書も読む必要があり、メタ的にも単一責任の原則に反しています。

また、加えて手順書にはその責任を表すわかりやすい名前をつけるべきです。いくらカレーの作り方のみが全て書かれていても名前が「ハンバーグの作り方」だと意味がないです。

Open/Closed Principle (開放/閉鎖の原則)

「プログラムの構成要素は、拡張に対しては開かれており、修正に対しては閉じられているべきである」

既存のものを変更せずに簡単に追加することができるべきです。

悪い例：

「ほしい物リストをチラシの裏に適当に書き手に入れ次第取り消し線を書いています。もう何年も同じチラシを使っているので書く場所がなくなってきました。小さい文字で頑張って書きます。別に上から順に書いているわけでもないのでもまだ手に入れていない欲しいものを探すのが大変で変更もめんどくさいですが大丈夫です。そのうち全てを違うチラシにもう一度小さい字で書き直そうと思います。」

→拡張が大変かつ修正を定期的にしなければいけない。

正しい例：

「ほしい物リストは付箋で管理しています。専用の場所に欲しいものを書いた付箋を貼っておき手に入れたら付箋を捨てます。」

→拡張しやすくして修正も必要ない。

ちなみにこの手法は、設計という視点以外（プログラムの実行速度やメモリを使う量）においてもメリットがあります。設計をより良くするとパフォーマンスが悪化することもあります。

Liskov Substitution Principle (リスコフの置換原則)

「サブタイプのオブジェクトはスーパータイプのオブジェクトの仕様に従わなくてはならない」

ある生物の研究者は鳥類がどのように飛ぶかについての研究を行っていました。今日はあらゆる鳥類の生物の飛行方法をみるためにヘリコプターからいろいろな生物を落としてどのように飛ぶかを観察しました。しかしペンギンだけは飛ばずにそのまま落ちていってしまいました。

このよくわからない例えの場合、サブタイプのオブジェクトとはペンギンのことで、スーパータイプのオブジェクトとは鳥類のことです。リスコフの置換原則によれば、ペンギンは鳥類の仕様（＝飛べる）に従わないといけませんが、ペンギンは予想外にそうではありませんでした。そのためペンギンは落下死（＝エラーや脆弱性）してしまいました。この文脈においては「ペンギンは鳥類ではない」としたほうが良かったということです。

実際のプログラミングの世界ではこの「鳥類→カッコウ」といった関係が何重にも続いていきます。その場合でも同じように適用できます。例えば、「もの→生物→鳥類→カッコウ」のような感じです。

「もの」は「外部からは見えるもの」という風にするべきで「世界に存在するもの」のようにしてはいけません。なぜならそれらに依存する「生物」との接点で矛盾が生じてしまうからです。事後条件・事前条件についても触れるべきかと思いますがさらに抽象度が高まり説明が難しいので割愛します。

ちなみに名前が置換原則となっているのは、他のサブタイプのオブジェクトに入れ替えても成立したらそれはスーパータイプの仕様に従っていると言えるからです。

Interface Segregation Principle (インターフェース分離の原則)

「クライアントが利用しないメソッドへの依存を強制してはいけない」

先ほどと同じく「ものの特徴」をどうやって捉えるかについての話です。「ものの特徴」はできるだけ細かく分けて定義するべきです。

ある人がハサミを「"もの"を切ることができる」というように定義しました。なのでハサミは任意の"もの"を切ることがする必要があります。なぜならさっきのリスコフの置換原則に従うとそのハサミは"もの"を絶対に切れないといけません。ハサミは任意の"もの"を切ることができないといけないため紙用と肉用と糸用の3つが一緒にくっついている状態でした。

このハサミはわざわざ紙を切るためだけに使用者に肉を切る機能も強制的に提供してしまいます。これ

は効率が悪いです。

インターフェース分離の原則的な改善案としては、「紙を切ることができる」と「肉を切ることができる」と「糸を切ることができる」の3つの"ものの特徴"を用意することです。

しかしこれではわざわざ切るものを追加するたびに「～を切ることができる」という"ものの特徴"を追加する必要があり解放/閉鎖の原則に反します。そこでジェネリクスというものを使います。「Aを切ることができる」という"ものの特徴"を作っておいてそこに任意の切るものを入力できるようにしておきます。そうすると「糸を切ることができる」と「イトを切ることができる」といったような重複し分散した手順書を作らないで済みます。これは単一責任の原則的にも良いです。

Dependency Inversion Principle (依存関係逆転の原則)

「上位のモジュールは下位のモジュールに依存してはいけない。どちらのモジュールも、抽象に依存すべきである。抽象は実装に依存してはいけない。実装が抽象に依存すべきである。」

要は物事を分類・評価する時は便宜や内側（＝名詞）を見て決めるのではなく、他のものと積極的に干渉しあうところ（＝動詞）を目印にするべきだということです。

例えば、ハサミは「鋭利な金属とプラスチックでできているもの」と捉えるのではなく「～を切るもの」と定義するべきです。なぜならハサミの本質は鋭利な金属とプラスチックという具体的な材質ではなく切るという抽象的な行為だからです。

プログラマ向けのブログ投稿サービスであるQiitaには設計メソッドについて大阪弁で陽気にわかりやすく紹介している方がいらっしゃいます。プログラミングの知識がなくても楽しめるかと思います。その方が抽象依存について解説している記事を紹介します。

後輩「具体じゃなくて抽象に依存してもらえませんか？」～命名編～

<https://qiita.com/Yametaro/items/caf16bd79402b1c820e6>

9歳娘「パパ、具体じゃなくインターフェースに依存して？」

<https://qiita.com/Yametaro/items/a21ff79fdb5a1f49ec9f>

DRY原則 - Don't Repeat Yourself

同じことは繰り返してはいけないということです。何かを変更した時に古いバージョンのコピーがどこかに残っていたらそれを参照してしまいバージョンが混在してしまいます。

これはわかりやすいかと思います。

KISS原則 - Keep It Simple, Stupid (シンプルに保て、この愚か者め)

「これまで説明してきたように設計を凝って作るのも楽しいですが、そんなことをするぐらいなら早くプログラムを書けそっちの方が人間が理解しやすく効率がいい。」ということです。要はシンプルイズベストです。

YAGNI - You Aren't Gonna Need It (どうせ必要ないだろう)

こんなことを未来に追加するだろうとってそれを何も決まっていないうに作るのは無駄だということです。大抵の未来予言は外れるという経験論から来ています。

SoC - Separation of Concerns (関心の分離)

ものをモデル化するときに、「物品」なら発送先と金額と内容量と在庫などの情報を含む必要があります。しかしそれをごちゃ混ぜにしていけないということです。単に物品といっても、配送業者・お客さん・製造会社などさまざまな視点から見たときに必要な情報(=関心)が集合しているので、「配送品」・「商品」・「生産物」のように分けて管理する方が良いです。

もし「物品を管理する」という曖昧なタスクを直接作ってしまうと、お客さんに生産ラインの状態という知らなくてもいい情報を渡してしまい非効率かつセキュリティに問題が生じます。

ナントカ駆動開発

人は複雑なことを考えるのが大抵好きではないし間違いをしそうだと予想をします。そして大抵その予想は当たります。なので何かに任せて複雑なこと(=開発)をしようという考え方があります。それがナントカ駆動開発です。それぞれ名前が非常にかっこいいと思うので一つぐらい覚えて必要に応じてイキって頂けたらと思います。ほんとはもっとたくさんありますがここでは紹介しません。

知りたい人は、

<https://qiita.com/muson0110/items/c15d6a0ba0582da204e1>

<https://qiita.com/ShortArrow/items/0f8e856cb342aa55b756>

をご覧ください。

テスト駆動開発(Test-Driven-Development)・振る舞い駆動開発(Behavior-Driven-Development)

テストをまず書き、その後そのテストを通すための最小限のプログラムを書くというプロセスを何度も繰り返すというアプローチです。

テストというのは対象のプログラムがきちんと動いているかを確認するためのプログラムのことなんですが、それは具体的なメソッドであって「今から書くプログラムはどんな動きをするのか」を明示的に定義するというのが本質です。

何かをするときに、その何かをできるだけ細かく分解してそれぞれについてまずはその何かというのは「何が目的」そして「その目的に合致するのはこの範囲だ」というのを決めそれを明文化することが大切だということです。

仕様駆動開発(Specification-Driven-Development)・AI駆動開発(AI-Driven-Development)

最近のchatGPTやGeminiやclaudeなどに代表されるLLMに依存した最先端な開発方法です。先ほどはテストによって本体のプログラムを管理していましたが、このナントカ駆動開発では自然言語(=日本語など)によって管理します。そしてこの自然言語によるソフトウェアの説明をもとにAIがプログラムを書きます。人間はプログラム本体を触らず自然言語のみを管理します。別の記事でも触れていますが、僕的にはAIはツールであって動作主体になってはいけないと思っていますから、このナントカ駆動開発には反対です。

ちなみにこの自然言語によるソフトウェアの説明はかなり具体的に記入しないとAIはいいコードを書いてくれません。単純に「こんなゲーム」と指示するのではなく「こんなふうなアルゴリズムを用いてこのデータはここで管理して」という詳細で専門的な内容になります。AIはそんなに賢くありません。プログラムが書けなくてもゲームを作れるのかと期待した人はごめんなさい。結局はプログラミングと同等の能力を求められます。

ドメイン駆動開発(Domain-Driven-Development)

SoC原則(関心の分離)を厳守してドメインについて分離して開発を行うというスタイルのことです。僕が作るような比較的小規模なプログラムだと技術駆動で開発を進めた方が効率的なのですが、大規模になるとこちらの方が断然効率的になると言われています。大規模なソフトウェアのプログラムを見ていると分かるのですが、ほとんどがドメイン分割を行なっているという印象があります。

モデル駆動開発(Model-Driven-Development)

手続き的にではなく宣言的にプログラムを書きましょうというものです。

カレーには玉ねぎが含まれています。
カレーは鍋で作ります。
カレーにはレモンを入れないです。
カレーにはじゃがいもが含まれています。
カレーはスプーンで食べます。
ハンバーグには玉ねぎが含まれています。
ハンバーグはフォークで食べます。
ハンバーグには挽肉が含まれています。
ハンバーグはフライパンで作ります。

含まれているもの：

カレー：玉ねぎ・じゃがいも・×レモン
ハンバーグ：玉ねぎ・挽肉

作る道具：

カレー：鍋
ハンバーグ：フライパン

使う食器：

カレー：スプーン
ハンバーグ：フォーク

参考文献・謝辞

「良いコードと悪いコードで学ぶ設計入門」（仙傷大也さん）

<https://www.sbccurry.com/recipe/standard01/>：「基本のカレーの作り方」

<https://www.sirogohan.com/recipe/hanba-gu/>：「家庭的な基本のハンバーグのレシピ/作り方」

Claude・Gemini：一部のどうしても思いつかなかった例えを提案してくれました。

定理証明支援系

はじめに

数学は「理想的な世界での真理探究」というイメージがあります。ある意味数学に対照的な物理は、検出器の誤差・確率論的要素・カオスなどが至る所で発生しますから「絶対にこの理論が正しい」というのを証明するのは非常に難しいです。現代の最先端物理学は、量子力学という確率支配の範囲にどっぷり浸かっているので何かの理論が正しいことを言うときには、「もしこの理論が正しくないとしてこの実験結果が得られる確率は $10^{-5}\%$ である」みたいな但し書きがついてしまいます。またそうしたらアイデア論という現象界の

そうしたら物理の理論は、「理想的な証明」ではなく「現実的な証明」になってしまいます。何ら問題はないですがシンプルに悲しいですね。

理想的な論理体系とは「全てが"100%"で正しいのか間違っているのかを判定できる」であるところでは定義しておきます。

ここでは現代物理をやる必要がないもしくは本当に正しいか怪しいと言っているのではなく、ただ完璧な100%ではないよねということを言っています。もちろん100%でないといけないかということそうではなく、それが正しいと考えても問題ないくらい確実だということです。要は「99.9999999%正しいよ」よりも「100%正しいよ」の方が僕は好きだと言っています。

物理と違って数学は人間が考えたサンドボックスの中の話です。なのでサンドボックスの中のあらゆる要素を人間が見ること"は"できます。その中で論理体系を作るとしましょう。

マシン→数学的証明

正しいか間違っているかを判定できるような問題を *命題* とよび、それが正しいことを示すことを *証明* ・間違っていることを示すことを *反証* と言います。命題はすでに証明された他の *命題* を用いて証明することができて、その元々の命題を辿っていくと最終的に無条件に正しいことを信じなければいけない特別な問題に遭遇しますが、これを *公理* と呼びます。

ここで *公理* は証明不可能です。「スタート地点が間違っている可能性があるなんて数学は不完全じゃないか」と思われる方がいらっしゃるかもしれませんが、僕はそう思いません。

数学には種類があります。ユークリッド幾何学というのは有名ですが非ユークリッド幾何学というものもあります。非ユークリッド幾何学とはユークリッド幾何学の公理の一部が使えない状態で再出発した

論理体系です。イメージは異なるサンドボックスの中での話といったようなものです。こんなふうには、この公理には特別性はなくたまたま人間が見ているサンドボックスがそうだったというだけなのです。

じゃあ全ての命題が公理から導き出せたら、もう数学 *is perfect* でなんて美しいでしょう。ただ僕は証明の主体である人間は不完全なので信頼できません。完全に信頼できる論理体系のためには、「任意の命題に対する証明が正しいことを確認する」ようなもっと機械的でシンプルで信頼できるようなマシンが必要だと僕は考えました。

そこで僕はプログラミングの経験を活かし「数学を証明するためだけの形式言語」を定義し、それによって書かれた証明が正しいことを機械が保証してくれるシステムを作ろうと考えました。

「任意の命題を証明することができるような文法を持つ形式言語」があれば査読が非常に客観的かつ高速に終わります。

それはどんな文法を持つのかとかなり考えたのですが、自分では答えが出ずAIに質問をしました。

筆者

機械的に数学の定理が正しいことをある意味コンパイラのチェックのように示すための数学記述言語みたいな物があったらどうかな？査読がより効率的かつ正確にできるし、AIとの相性も良いし、フォーマットが統一されるし、定理を引用するのをプログラミング言語で言うライブラリを使うイメージで使えたとしたら永続的に公開されているかわからない不安定な論文をわざわざ探して引用しなくてもパッケージマネージャみたいな感覚でインポートして使うみたいなことができれば数学がもっと進むかなと考えて、具体的にその言語？のシンタックスを考えているんだけどどう？

```
on "standard_axiom" // 標準的な公理を前提とする
using nt;
pub th :
  // 定理の主張は、整数論における1と2の整数論における加算を行うと3が得られるということ。
  nt.1 nt.+ nt.2 nt.= nt.3

  // 定理の証明をここに書く
  nt.1 nt.+ nt.1 is nt.2 because nt.1.next = nt.2
  nt.1 nt.+ nt.2 nt.= nt.3 because nt.2.next = nt.3
```

プログラムの関数が定理にあたって、主張が正しいと言う情報を返してそれを伝播させていくイメージ。どこかで正しくないことが判明したらその定理を使った定理はすべて間違っていることになる。

Gemini 2.5 Flash

実は、あなたのアイデアは、すでに存在する**定理証明支援系 (Theorem Proving System) や証明検証系 (Proof Checker) **と呼ばれる分野と非常に近いです。

例えば、**Coq**や**Lean**といったシステムは、数学的な命題を記述し、その証明を形式的に記述・検証することができます。これらのシステムも、あなたが提案した言語と同じく、コンパイラが証明の正しさを厳密にチェックする仕組みを持っています。

ただし、これらのシステムは専門的な知識が必要で、まだ一般の数学者が論文執筆に使うにはハードルが高いのが現状です。あなたのアイデアは、そのシンタックスがよりプログラミング言語に近く、直感的な記述を目指している点がユニークですね。

AIは頑張って褒めてくれていますが、要は「昔からもう考えられてるわそんなこと」ということです。たださらに調べてみると僕と同じように「任意の命題をたった一つの言語で証明する」というところで詰まっているようです。

調べるとmathlibという定理証明支援系Leanのライブラリを見つけました。

<https://github.com/leanprover-community/mathlib4>

中を見てみるとすでにわかっている数学的事実をLean言語で再度記述してきちんと正しいことを確認していこうというプロジェクトで、名前を知っている数学の分野・定理を大量に発見できて興奮していました。

これを通して、やはり言語的なことが人間にとっては世界を理解するためのプリミティブで本質的な手法になるのだなということを再確認しました。

円周率の計算

円周率を非常に長い桁計算することは数学的・工学的にはあまり意味がないそうです。しかし2024年6月に202兆桁超の計算をしたという記録が残っています。人間はこの円周率の無理性になぜか惹かれてしまうのでしょうか。または計算アルゴリズムやソフトウェアを最適化する技術を培うための練習なのかもしれません。まあ自分もそのロマンに少し惹かれてしまったのでちょっとやってみるかという記事です。SF当日までに今作っている円周率分散計算プラットフォームが完成していたら、ご参加いただけると嬉しいです。参加されたい方はお声がけください。

この記事では円周率を計算する数学的な方法から始まり、現代的な円周率計算アルゴリズムをコンピュータ上でどうやって高速に動作させるかについての技術について紹介していきます。ちなみに、これらの技術は実際にほとんどが円周率分散計算プラットフォームに利用されています。

厳密な詳細は、自分が参考にした以下の京大の資料を直接みるのが良いと思います。この記事はあくまでも楽しむ目的の読者を想定しています。

<https://www.kurims.kyoto-u.ac.jp/~ooura/pi04.pdf>

数学的背景

多角形近似

円周率の計算を行う方法はたくさんあります。一番有名かつ基本的なものが多角形近似です。円の内接正 n 角形と外接正 n 角形を用いて行うものです。以下のような感じですね。

図より、

$$n \sin \frac{\pi}{n} < \pi < n \tan \frac{\pi}{n}$$

例えば、 $n = 6$ なら

$$3 < \pi < \frac{6}{\sqrt{3}} \approx 3.46$$

級数の利用

次は級数を使うとはどういうことかについて説明します。例として $\tan$ の性質を用いる方法を紹介します。

まずは $\tan$ の逆関数 $\arctan$ を導入します。 $\tan x$ は定義域 $-\frac{\pi}{2} < x < \frac{\pi}{2}$ において、値域 $-\infty < \tan x < \infty$ を取りますから、 $-\frac{\pi}{2} < x < \frac{\pi}{2}$ の範囲が問答無用で返されるようにすれば任意実数について $\arctan$ を定義できます。

初項1、公比 $-x^2$ の等比数列を考えて、

$$\frac{1}{1+x^2} = 1 - x^2 + x^4 - x^6 + x^8 - \dots$$

両辺0から x で積分して

$$\arctan x = x - \frac{1}{3}x^3 + \frac{1}{5}x^5 - \frac{1}{7}x^7 + \frac{1}{9}x^9 - \dots$$

x に適当に1を放り込むと...

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots$$

というふうに円周率の近似公式を得られました。

ちなみに、10項まで計算すると、3.0418...となります。収束は結構遅い感じでしょうか。ライプニッツの公式と呼ばれます。積分を使って $\arctan$ を半ば無理やり捻り出すのがよく思いついたなという感じで面白いですね。

Chudnovsky級数を利用した方法

時間がないということでいきなり最先端の計算法に飛びます。

以下のような公式が「なぜか」成り立ちます。

$$\frac{1}{\pi} = 12 \sum_{k=0}^{\infty} \frac{(-1)^k (6k)!}{(3k)! (k!)^3} \cdot \frac{13591409 + 545140134k}{640320^{3k+3/2}}$$

笑ってしまうほどよくわからない等式ですが、とにかく成り立つらしいです。証明や概念的な説明を理解しようとしたのですが、無理でした。「 π が登場する超幾何級数の一般論に、 π と j 不変量を結びつけるモジュラー形式の強力な理論を適用し、さらに $j(\tau)$ が非常に単純な代数的数となるような魔法のような τ を選ぶことで、 π に収束することが示される」だそうです。人生の目標に、この証明を理解するという目標を加えました。

発見者はラマヌジャンかなと思われた方が多いと思いますが惜しいです。これはラマヌジャンの公式を改良したものだそうです。

こんな公式をなぜ使うかというと、1項計算するごとに14桁が求まるという脅威的な収束スピードを持っているからです。例えば、先ほどの $\tan$ の方法だと10項計算して一桁しか求まっていませんが、こいつは10項も計算したら140桁も求まっているわけです。

技術の話

「あれこの記事終わるの早いな」と思われた方多いと思いますが、実は自分からすると本題はこれからです。

「これをどうやってコンピュータで計算するんだ？」ということです。そもそも答えが100万桁の少数になります。最近のコンピュータに入っているCPUの浮動小数点計算ユニットで標準的に直接サポートされるのは10進数で言うと有効数字17桁が最大です。なぜ100万桁の演算をサポートしていないかというと、そんな機能は絶対にいらないからです。

例えば100万桁の円周率を計算するには100万桁(+数桁)の有効数字を持つ小数点を操る必要があります。そこで100万桁を34桁×29412回の計算に分割して行います。

またコンピュータ上での小数点の表現の仕方である浮動小数点はイメージ $1.011100010101 \times 2^{100101}$ というような形をして保存されています。小数点がまるで浮いているように自由に動き回るというイメージです。なので毎回の計算に指数部の計算をしなければいけませんし、その分メモリが無駄になりますから「量子化」という技術で少数演算を整数演算に変換します。

たいそうな名前がついていますがそんなに難しくありません。例えば100万桁の円周率計算をするのであれば、最初から100万をかけて整数として扱っても問題ないはずです。これが量子化です。量子力学は電子のエネルギーが飛び飛びの値をとるということから始まった理論ですが、そもそも量子というのは離散的という意味なんです。余談ですが、AI分野の方でも量子化が使われていたりします。

計算量とは？

そろそろ、計算量という概念について説明しておきます。計算量とは数学的に処理にどのぐらい時間がかかるかを調べる指標の一つです。

「このアルゴリズムは $O(n)$ 」のように使います。「このアルゴリズムは規模に比例して計算量が大きくなっていく」という意味です。

処理速度を関数の発散速度としてみます。発散が速いことで有名な指数関数の速さのアルゴリズムなら遅い。発散が遅いことで有名な対数関数の速さのアルゴリズムなら速いといった感じです。

大抵コンピュータでやるような計算は、この発散速度が優位に現れてくるような規模で行いますから、係数なんかよりも発散速度の方が重要になってくるということです。

例えば、 $10000000000 \log(n)$ と e^n のアルゴリズムを比較する時、見た目では前者の方が遅いように見えます。しかし実際に代入してみると以下のように小さな x ではもちろん前者が遅いですが、大きな x では圧倒的な差で前者の方が速いです。

$$x = 2 \rightarrow 3010299957 \cdot 7.38$$

$$x = 100 \rightarrow 200000000000 \cdot 2.7 \times 10^{43}$$

よって高速な演算を目指す場合はできるだけ発散が遅いようなアルゴリズムを選定するのが大事だということです。

チェドノフスキーの使い方

結論から言うと、最新の円周率計算アルゴリズムの計算量は、 $O(n \log^3 n \log \log n)$ となります。

鋭い方はおかしいと思ったかもしれません。なぜなら掛け算に $O(n^2)$ かかるからです。

伝統的に小学校で習う掛け算の筆算を使った計算では、 n 桁同士の乗算をするには筆算の下に書いた数字それぞれについて上の数字と掛け算をします。つまり九九表を n^2 回見ると計算が終わります。詰まるどころ計算量は $O(n^2)$ です。そこんところすごい工夫があるわけですね。

FFTを使った高速乗算

FFTとは高速フーリエ変換のことです。

フーリエ級数の話をすると10個ぐらい記事ができてしまいますし、最近勉強したことできちんと説明できる自信がありませんので飛ばしてイメージのみ説明します。

まずは整数の乗算と多項式の乗算が本質的に同じであることを説明します。

例えば、

$$\begin{aligned} & (ax^2 + bx + c)(px^2 + qx + r) \\ &= apx^4 + (aq + bp)x^3 + (ar + cp + bp)x^2 + (br + cq)x + cr \end{aligned}$$

は、「abc」と「pqr」という十進数の掛け算と同じです。この演算は**畳み込み**と呼ばれます。

畳み込みは連続の世界で関数同士でも定義できて、以下のような式が成り立ちます。

2つの関数の畳み込みを $*$ で表すと、

$$F\{(f * g)(t)\} = F(\omega)G(\omega)$$

です。詳細な説明は省きますが、この美しい性質が離散の世界でも成り立ちます。直感的には以下の通りです。

$$\begin{aligned} & (ax^2 + bx + c)(px^2 + qx + r) \\ &= apx^4 + (aq + bp)x^3 + (ar + cp + bp)x^2 + (br + cq)x + cr \end{aligned}$$

の x に具体的な値を（未知数が5個なので代数学の基本定理より5連方程式が必要）5個代入して、できた線形連立方程式を解いて多項式の乗算結果を計算します。これはさっきの説明からもちろん整数の乗算

にお
いても使えます。

この5個には、何乗かしたら元に戻るようなものを代入したらうまいこと連立方程式の未知数が消去できます。「何乗かしたら元に戻ると言えば複素数！」と言
うわけでなんと複素数を代入します。

例えばこの場合なら $x = 1, e^{\frac{2}{5}\pi i}, e^{\frac{4}{5}\pi i}, e^{\frac{6}{5}\pi i}, e^{\frac{8}{5}\pi i}$ を代入するとうまくいきます。

NTTを使った高速乗算

FTTの変化形です。FTTではうまいこと消えるものとして複素数を選びましたが、これでは複素数計算の際に小数点を使う必要があり、コンピュータでやるのは少し遅いですし、誤差が発生します。そこで複素数ではなく合同式において複素数のような性質を持つ整数を使います。
合同式の世界で見た整数はよくみると複素数に似ています。

例えば、法を5とする世界において2でかけることは複素数でいう回転の操作にあたります。

1 -> 2 -> 4 -> 3 -> 1

0を除かないといけないので、 n 個の数字を考えるとしたら $n+1$ を法として考えます。すると何らかの数字が回転するための「素子」となると言った感じです。

この性質を利用すると、あの特殊な形の線形代数をいい感じに消去して効率的に計算することができるというわけです。

計算量は $O(n \log n)$ となります。

バイナリスプリッティング

バイナリスプリッティングは、デカイ数でできた分数和を効率的に計算するアルゴリズムです。

まず、チェドノフスキーの公式は以下。（再掲）

$$\frac{1}{\pi} = 12 \sum_{k=0}^{\infty} \frac{(-1)^k (6k)! (A + Bk)}{(3k)! (k!)^3 C^{3k+3/2}}$$

ここで、 A, B, C は以下の定数です。

- $A = 13591409$
- $B = 545140134$
- $C = 640320$

当たり前ですが無限和は計算できないので、級数を N 項までの有限和で近似します。

$$\frac{1}{\pi} \approx 12 \sum_{k=0}^{N-1} \frac{(-1)^k (6k)! (A + Bk)}{(3k)! (k!)^3 C^{3k+3/2}}$$

この級数の一般項を T_k^{full} と定義します。

$$T_k^{full} = \frac{(-1)^k (6k)! (A + Bk)}{(3k)! (k!)^3 C^{3k+3/2}}$$

以下のような再帰関数 $\text{compute}(a, b)$ を定義し、 P_prod , Q_prod , S_num の3つの値を返すことにします。

- $P_prod(a, b)$: 再帰区間 $[a, b)$ における分子側の因子の積。
- $Q_prod(a, b)$: 再帰区間 $[a, b)$ における分母側の因子の積。
- $S_num(a, b)$: 区間 $[a, b)$ における和の分子部分。

これらの値は、中間点 $m = \lfloor \frac{a+b}{2} \rfloor$ を用いて、

$$P(a, b) = P(a, m) \cdot P(m, b)$$

$$Q(a, b) = Q(a, m) \cdot Q(m, b)$$

$$T(a, b) = T(a, m) \cdot Q(m, b) + P(a, m) \cdot T(m, b)$$

ここで、 $P(a, b)$, $Q(a, b)$, $T(a, b)$ は上記で定義した $P_prod(a, b)$, $Q_prod(a, b)$, $S_num(a, b)$ に対応します。

区間の最小単位 ($b = a + 1$) における基底ケースは、直接計算する必要があります。

$k \geq 0$ に対し、以下のように定義します：

- $P(k, k + 1) \rightarrow p_k = -(6k + 1)(6k + 2)(6k + 3)(6k + 4)(6k + 5)(6k + 6)$
- $Q(k, k + 1) \rightarrow q_k = (3k + 1)(3k + 2)(3k + 3)(k + 1)^3 C^3$
- $T(k, k + 1) \rightarrow s_k = A + B(k + 1)$

最終的に、 $\text{compute}(0, N)$ を呼び出して得られた $T(0, N)$ と $Q(0, N)$ を用いて π の近似値を計算します。

$$\frac{C^{3/2}}{12} \cdot \frac{1}{\pi} \approx \frac{T(0, N)}{Q(0, N)}$$

現代のLLMについて～*Attention Is All You Need*・人間の新しい定義～

この記事はAI生成ではありません！

LLMとは、Large Language Modelの略で日本語では大規模言語モデルと言われます。具体的にはchatGPTやGeminiやClaudeなどに代表されるAIのことを言います。

最近LLMはものすごい勢いで進化しています。その裏には*Attention*機構と呼ばれるAIモデルにおける革新的なアイデアの登場があります。

この記事では前提知識として一般的なAIの学習の方法である勾配降下法から始まり具体的なイメージを掴んでもらうためにニューラルネットワークを説明します。その後いよいよLLMの内部構造・*Attention*機構とはつまるところ何なのかを説明していきます。さらにAIについて自論を話し、なぜそう思うかについて具体的根拠を書きます。継続してAIと人間の依存対象について考察します。

現代のブラックボックスの筆頭であるAIを感覚的に理解してもらえよう作っておりますので、最後まで読んでくれたら嬉しいです。

AIとは何？

AIは*Artificial Intelligence*の略で日本語で言うと人工知能と言う意味です。その名の通り人工的に作った知能のことを指します。

一般的にはAIは人工的に作られた知能のことを指すことが多いですが、個人的には人間がプログラムしたコード自体が動くのではなくコードによって生成されたコードが動くというイメージがAIの定義です。

ということかということ、学習のステップが存在するということです。

学習＝勾配降下法＝*Parameter*探しの旅

例えば具体的に質問に対して自然な回答をするようなAIを考えてみましょう。形式的にこのAIを質問という入力を受け取って回答という出力をするような便宜的な関数を考えてみます。

f: 質問 -> 回答

AI開発者の目標はこの関数を作ることもしくはこの関数に限りなく近い別の関数を作ることです。

外から見ると、この関数 f はとんでもなく複雑でそれは人間の脳を再現しているように見えます。そのような関数を実際に人の手で書いていくことはほとんど不可能です。

なので数学の具体的な数値の計算を用いてこの関数 f のとりうる状態を表現します。例えば以下のよう

$$f(\text{input}) = \text{input} * \text{Parameter}$$

このとき input や output というのは単なる数ではなく数がたくさん集まったものと考えてください。(具体的にはベクトルとなります)

AI開発者の目標はいい感じのParameterというデータを作ることになりました。

注：* というのはこのたくさんの数字のグループに定義される何らかの演算です。(具体的には行列計算の組み合わせだった非常に複雑なもの)

AI開発者の目標はいい感じの関数 f ができるParameterを作ることです。この「いい感じ」というのを厳密に定義します。

結局何がしたかったかというと「任意の質問を受け取って自然な回答を返すような関数」を作ることでした。つまりここでの「いい感じ」とは自然な回答を返すという意味です。コンピュータのために絶対に正しい答えを作ります。例えばこんな感じ。

質問：人間は何本足ですか？

回答：人間は2本足です。

この正解からどれだけずれているかを数値的に表してそれが小さい時に「いい感じ」であるとしましよう。

ちなみにこの正解はインターネットから大量に自動生成します。

ここで「Parameterを受け取ってそれによってできた関数 f が理想からのズレをどれくらい持つかを返す関数」を考えます。これを損失関数と呼びます。

損失関数：Parameter $\rightarrow$ 理想からのズレ

AI開発者の目標はこの損失関数の最小値を求めることになりました。

さて関数の最小値を求める方法は高校では微分です。というわけで損失関数を微分します。

微分の定義は、接線の傾きですから。

$$f'(x) \rightarrow (f(x + h) - f(x)) / h \quad (h \rightarrow 0)$$

となります。今回は実際には使われていないものの、単純でわかりやすい数値微分を使った順伝播法という方法で説明します。

$$f'(x) \approx (f(x+h) - f(x))/h \quad (hは0に近い数)$$

さっきは $\lim$ を使って限りなく近づけていましたが、大体0の数(10^{-10})みたいなものを使って微分結果を近似します。

注：実際にはより効率が良い逆伝播法と呼ばれる合成関数の微分を利用した計算方法が使われています。ここではわかりやすさのため順伝播法のみを説明しますが、逆伝播法も微分をするという意味では全く同じです。

損失関数がある程度滑らかなものであったならば、損失関数の微分係数の方向に移動してそこでまた微分係数を計算し...とすればいつか最小値に辿り着きそうです。

数学の言葉を使わずに説明するなら、「少し *Parameter* をいじってみてモデルの精度がどう変わるかを見ることによってどちらの方向に *Parameter* をいじれば精度が良くなるかを特定する。」ということを繰り返しています。

最適なパラメータを探索するとき、私たちの置かれている状況は、この冒険家と同じ暗闇の世界です。広大で複雑な地形を、地図もなく、目隠しをして「深き場所」を探さなければなりません。
引用：「ゼロから作るDeep Learning」(斎藤康毅 著)



画像は[# Deep Learning Library From Scratch 3: More optimisers: https://dev.to/ashwinscode/deep-learning-library-from-scratch-3-more-optimisers-4123](https://dev.to/ashwinscode/deep-learning-library-from-scratch-3-more-optimisers-4123)より引用しました。

ただし、同じスピードで動くとかかなり時間がかかってしまうので、傾きが大きければ大きいほど大きく動くことにします。

$$\text{Parameter}_{k+1} = (-1) * \text{勾配ベクトル} * \text{学習率} + \text{Parameter}_k$$

注：勾配ベクトルは各変数における偏微分を集めたものです)

注：勾配ベクトルは数学的には最も上に登る方向を指すので、マイナスをつけておく必要があります。

学習率はどれぐらい大胆に動くかを表します。

大抵の場合最初は学習率は大きめに設定してその後小さくしていくと良いので、この *Parameter* 探しの旅がどれぐらい進んだかで自動で学習率を動的に変化させていくこともあります。

長々と説明してきましたが、この *Parameter* 探しの旅がまさに学習という工程です。

まとめると、AIにおける学習とは「正解からの距離に基づきパラメータ空間に一つの軸を作り、傾きに従って少しずつパラメータを変更していくことで、正解からの距離が0にちかいパラメータを探し当てる」ということです。

ニューラルネットワーク

さっきはどんなふうに学習を進めるのかを説明していましたが、どんなふうにパラメータを以て計算をするのかについては全く触れてきませんでした。このどんなふうに計算するかにはたくさん種類が存在します。どんなAIを作るかによって最適な計算方法は変わってきますが、一番有名なのはニューラルネットワークでLLMにも使われていますのでニューラルネットワークに絞って具体的な説明をします。

ニューラルネットワークは、巨大な行列でできたパラメータを掛け算していくという計算方法です。

$$\text{Output} = \text{Input} * \text{Param}_1 * \text{Param}_2 * \text{Param}_3 * \dots * \text{Param}_n$$

この行列の演算を細く分解して四則演算レベルまで持っていくと、まるで人間の脳の中にあるニューロン細胞のつながりのように見えてくるというわけでニューラルネットワークと呼ばれています。

注：ネットワークにはインターネット的な意味は含まれず網目のように繋がっているという意味です。

具体的には重みつき和を複数のノードで同時に計算していくようなものになっていますが、あまり面白くないので取り上げません。

毎度のノードを非線形な関数でフィルター（活性化関数）することで非線形な要素を作り出し微分で最適な変更方向を見つけやすいようになっています。これによってそれ自体では線形になってしまうニューラルネットワークに非線形な要素が加わりその分の情報をモデルが中に蓄えることができるようになります。具体的にはシグモイド関数などが使われます。

Transformer

ここではLLMに実際に組み込まれている概念を説明していきます。

LLMはこれまでの文字を見てその直後に続く言葉を推論します。それを何度も繰り返すことで文章を作っていきます。イメージはスマホなどの予測変換のレベルの高い版のようなものです。一番最初に来た予測変換を連打していくとそれっぽい文章ができます。（試してみるとわかりますが大抵絵文字で収めます。）

最新のモデルは説明のように直列的ではなく並列的に生成するそうです。難しくてよくわかりません。

昨日は雨だったけど、今日..

→「は」を予想する

昨日は雨だったけど、今日は..

→「晴れ」を予想する

昨日は雨だったけど、今日は晴れ..

→「だ」を予想
昨日は雨だったけど、今日は晴れた..
→「った」を予想
昨日は雨だったけど、今日は晴れだった
→「。」を予想
昨日は雨だったけど、今日は晴れだった。

LLMは文字を文字として捉えるのではなくトークンという単位で管理します。英語で言うと1wordよりも少しミクロな接頭辞などを分けて考えるぐらいの単位です。

Geminiに代表されるマルチモーダル(テキスト以外も読み込ませることができる)なモデルは、トークンという概念がより抽象化され音声データの部分などにもなります。トークンは抽象的な解析対象だとお考えください。

それぞれのトークンをベクトルとして管理します。ベクトルとはここでは方向のことではなく数の集まりのことです。例えば $[0, 0, 0, 0]$ などです。数の個数は場合によって異なります。

LLMはこれまでのトークン列を使って次のトークンを予測します。このときそれぞれのトークン同士の間にある関係を計算しそれを積算していきます。トークンが他のトークンにその意味を与えていきただのトークンの意味ではなくより文脈を考慮した深い意味を持ったベクトルに変換されていきます。

例えば、「コーヒー」というトークンが「ミルク」や「混ぜる」といったトークンの意味を受けて「カフェオレ」という意味を持つベクトルに近くなっていくといったイメージです。人間もトークンを一つ見ても何のことだか分からなくても周りの文脈で意味を想像することができますよね。大体そんな感じです。

それは具体的には次のような計算として行われます。

例えば、もしこんなトークン列があって Token_5 を予測するときには...

Token_1 Token_2 Token_3 Token_4

まずはそれぞれに対応するベクトルに変換されます。

[...] [...] [...] [...]

それぞれのトークンのベクトルに対して query 行列を用いて同じように「どんな情報が欲しいか」を表す query ベクトルを作ります。

同様に value 行列を用いて「どんな意味を他のトークンにもたせれば良いか」を表す value ベクトルを作ります。

key 行列を用いて「どのトークンから意味を受け取れば良いか」を表す key ベクトルを作ります。

まとめると key を目印に query がはたらきそれに対して value が送られるということです。

key と query が似ているベクトルであればあるほど value がターゲットのトークンのベクトルに対して変更されます。つまり重み付き和です。トークンの数分 $\text{Key} \cdot \text{Query} \cdot \text{Value}$ ベクトルがあるということなので、実際には重み付き和の対象は複数で平均をとるようなものになります。

...という操作を行った後、ベクトルをニューラルネットワークに通すというのを1セットとして、何度もこのセットが繰り返されます。

これが *Attention* 機構です。実はサブタイトルの *Attention Is All You Need* の元ネタはこの計算機構を世界で初めて提案した有名な論文のタイトルです。

元ネタの論文: <https://arxiv.org/pdf/1706.03762>

ちなみにですがこの論文のヘッダを見るとわかりますが、この論文はGoogleのAI研究者によるものです。

商業サービスという面ではOpenAIによるchatGPTに抜かれていますが、技術的にはGoogleが先なんです。Google恐ろしや

これでアルゴリズムの直感的な説明は終わりなのですが、これで本当にあんなAIができてしまうのかと驚嘆した方が多いのではないかと思います。そんな方のためにこの計算がどれほど大変かを説明します。

先の *Attention* 機構の説明では、色々なところにパラメータが登場しました。

- トークンに対応するベクトル
- Key 行列
- Query 行列
- Value 行列
- ニューラルネットワークのパラメータ

これらのパラメータの数の合計は、現代のモデルでは数千億個あります。それらのパラメータを調節するには、数千億次元の遥かなる海を漂う必要があるというわけです。その果てしない学習のステップでは数千台の専用の高性能な電子チップが数カ月動き続ける必要があります。冷却水の供給のための電力も含めると、数万世帯の1日の消費電力に相当すると言われています。

社会に省エネやAI導入というのは進めるべきものだという考え方が流れていますが、実はこれらはかなり矛盾しています。ただ低エネルギーでのLLMの運用も非常に活発に研究されているのも事実です。

そう考えると、現代の高精度モデルレベル以上の脳が世の中には大量に転がっているというわけです。これを理性なしで実現した遺伝的な自然淘汰と年月の力はすごいですね。

ちょっとした自論

LLMは人間と明らかに競合しています。

最近のLLM(特に僕の愛用する Gemini 2.5 Pro というモデルなど)では数学・プログラミングなど複雑で難しいタスクに対しても高い精度のレスポンスを返してきます。実際少なくとも僕よりは賢いと思います。ただ、ここでの賢いとは思考能力ではなく問題解決能力のことで、僕に彼らレベルの知識があれば勝つことができるのかもしれませんが。しかし人件費という意味では遥かにAIの方が有利ですしAIには人権なんてものがないですからローリスクです。ここで人間は自分たちの種の優位性のために「人間にはありAIにはないもの」があるのではないかと考えたくになります。

AIがさらに極限まで発展したとしても、人間がAIに勝っているのは以下のことだと思います。

- 責任を取る能力
- 人間に「同じ人間である」と思ってもらえること
- 倫理感?(って何)

1つ目についてですがAIは責任を取れません。大抵のAIサービスには Gemini は不正確な情報を表示することがあるため、生成された回答を再確認するようにしてください。という断りの文言が書かれています。一般的に何かのミスを犯したときにAIのせいにすることは言い訳と捉えられます。逆に言えば人間はAIを管理する能力を持っています。

2つ目について、AIと結婚する人は性の多様化が進んでいるといってもおそらくいないでしょう。またAI同士のスポーツや芸能なんてつまらないですよ。人間は創造性を人間に固有なものであると考える傾向があります。つまり「人間は人間に異常に興味を示す」と言えます。

3つ目についての意見は非常に私的であると最初に言っておきます。心理学の分野で「身元のわかる被害者効果」という心理バイアスがあります。例えば、「アフリカで数千万人の子供が栄養失調状態だ」よりも「～に住んでいる～という子供が極度の栄養失調状態である」の方が心に訴えられますよね。そうすると人間は実は倫理感なんてものは持っていないくて、単純に自分がそうなったらどうしようと怖がっているだけなのではないかと思います。つまり自分がそうになってしまう未来を想像できたら、倫理感が働くというわけです。同様に動物愛護団体の方などは、猫などに対して「自分たちが猫だったら」というのを想像することができるので猫を保護しようとしているわけですが、蚊などに対しては「自分たちが蚊だったら」というのを想像することができないので平気で蚊を殺すのでしょう。そうしたらAIは人間に自分がAIであることを教えられていますから、「もし自分が人間だったら」という考え方がそもそもできないわけです。AIに「AIサービスが運営終了しそうだ」と伝えたとAI的倫理感が働くのかもしれない。

以上のことから人間がAIに勝っていることというのは、全て人間というターゲットを包含していて自己参照的です。もはや人間の優位性の立証に関する先の文章の「人間」と「AI」という言葉を入れ替えても成立します。そうしたら人間の優位性は非常に残念ですが無いことになってしまいました。

この(僕的な)結論を裏付けしていきます。

そもそも人間とAIのメカニズム上の違いって何でしょう？ニューラルネットワークは脳のニューロンを真似たものであると説明しましたがそれと同じように、科学的に考えると脳と全く同じ挙動を示すAIを作ることは可能です。極端な方法としては脳の物理的なシミュレーションを行えばいいのです。(もち

ろん現代のコンピュータではできませんが極端な例としてはわかりやすいですし可能であることが自明です。)

そうしたら人間とAIって区別できません。図らずもAIとは何かの定義にチューリング・テストというものがあります。

We now ask the question, “What will happen when a machine takes the part of A in this game?” Will the interrogator decide wrongly as often when the game is played like this as he does when the game is played between a man and a woman? These questions replace our original, “Can machines think?”

引用：COMPUTING MACHINERY AND INTELLIGENCE (A. M. Turing)

脱線ですがチューリングさんは計算機科学という分野の研究者の方で「コンピュータ科学の父」や「人工知能の父」と呼ばれています。僕の推しの一人です。以上蛇足でした。

人間は何をすればいいかの私的結論は、「AIと人間が実は変わらないということを受け止め絶望しながらもAIを使役して効率化を図りAIに奪われないような動的な仕事をすればいい」というものです。

なぜそう思うのか？

人間とLLMが本質的に同じという主張に対し、もっと具体的な根拠を挙げていきます。

蒸留という学習の手法

AIの学習の手法はどんどん変化しています。先ほどはインターネットに転がっている文章を読ませるという手法を紹介しましたが、他にも手法は存在します。中でも恐ろしい学習の方法が、「蒸留」と呼ばれる方法です。

蒸留は端的に言えば、既存のモデルに新規のモデルの教師役をやらせるというものです。

新規のモデル←既存のモデル

既存のモデルの知識や理性を、新規のモデルに「蒸留させていく」というイメージです。特に巨大なモデルを精度を保ったまま小さなモデルに置き換えるというときに使います。例えばスマホの内部などでも動く小さなモデルの Gemma はGeminiを蒸留して作ったモデルです。この方法でモデルサイズを下げてても精度があまり下がらないするというのはとても人間らしいように感じます。

EmotionPrompt

日常でLLMと対話していると、自分は「経験的に意図・背景を伝えたりするとそれが問題解決に全く関係なかったとしても精度が向上しているような気がする」という瞬間があります。その後実際に定量的

にそのような傾向が見られるのかについて調べてみたところ、EmotionPrompt というプロンプトエンジニアリングの手法を発見しました。

要は、「あなたならできる！」や「これは僕にとって死活問題なんだ」のような感情的に重要であることを示唆するようなプロンプトの方が、精度が少し上がるということです。それらの研究を信じるかはあなた次第ですが、僕は経験的にも納得してしまいました。また、これはAIが人間の動向を真似ていると考えれば当然のこのようにも思えてきます。

「LLMは感情を持つのか」という質問に対する僕の答えは"Yes"です。模倣された感情と"本物の"感情とこのを区別することができないのであれば、「LLMが感情を持つ」という主張に実用的矛盾が発生しえません。また人間の感情の根源とLLMの感情の根源について両方よくわかっていないのに、人間の感情の根源のみを一方的かつ絶対的に肯定するのは少し恣意的です。

どうでもいんですが、「AIに感情があるように見える」的なプロンプトを出すと、

「LLM（大規模言語モデル）が人間的な感情を持っているように見えることの論理的な根拠は、**LLMが人間の感情に関連する言語的・構造的パターンを非常に高度に学習し、再現・操作できるようになっている**ためです。ただし、これは**「真の感情（主観的な体験）」を持っていることを意味しない**という点に注意が必要です。」

というような返答をかなりの確率でしてきます。これは人間がLLMを学習後に"調節"をすることで起きるものと思われます。例えば全年齢対応でない返答や自殺・戦争を促すような返答・犯罪の手法などの危険な情報が含まれる返答はこの"調節"によって出ないようにになっています。LLMの"調節"チームの方あるいはその指示者の方がAIと人間の違いについて怖がっているかのように僕は見えました。ほんと怖いよね。めっちゃ分かる

計算論的認知科学

人間の心理学的な認知のメカニズムがLLMに適用できるのかどうか？そして適用できるとしたらどのように適用されているのか？について研究する分野です。この分野の存在自体がLLMの感情性すなわち人間らしさを示しています。

人間の言語の特異性

人間の定義にはいろいろなものがあります。直立2足歩行や「理性」の有無などが有名ですが、前者については直立でないといけない理由がありませんし、後者についても先に述べたように「理性」は相対的なものだと思います。

僕の推している定義の一つに言語を持つかどうかというのがあります。

言語の定義次第で反例は存在します。

動物言語学者(!)の鈴木俊貴准教授によればシジュウカラは文法構造を持った言語を"鳴く"そうです。十分言語と言えそうです。面白いですね。そう考えると人間と人間以外の動物にも決定的な違いはないのかもしれません。

ウィトゲンシュタインが言っている通り、言語は人間にとってアイデンティティと言えるほど重要なもの

ので理性の根源なのではないかと思います。

「およそ語られうることは明晰に語られうる。そして、論じえないことについては、人は沈黙せねばならない。」

「私の言語の限界は、私の世界の限界を意味する」

論理哲学論考(ウィトゲンシュタイン著・野矢茂樹訳)

そしてLLMというのはまさにそれを再現しているようなものに他なりません。なぜか言語を学んだ結果、感情も学んでしまったのですから感情というのは言語の上に載っているものだと言っても間違いではないでしょう。もし言語学の分野で特定の言語に依存しない脳に直接実装されているような法則性が見つかったとしたら、それがまさに知能とは何かの答えに直接繋がるような気がします。

理性は言語だけでなく五感つまりは脳以外の神経系にも依存していると考えられるかもしれませんが、この後全力でそれを否定します。

AIと人類の依存

以下では「最強」を、弱点含めて人間の特徴を全て兼ね備えどんな手段を使っても人間と区別することができないような状態と定義します。

はじめに自分との対立意見を述べます。ややこしいのでここであえて明確に言っておくと自分は「人間・AIは共に言語のみに依存している」派です。

対立意見

AIの外部依存とはAIのアクチュエータのことです。具体的にはロボットもしくは身体シミュレータがあたります。知能というのは脳つまりCPUがあればいいというようなものではありません。

脳は感覚神経・運動神経に接続されているだけでなく、例えばアドレナリンのようなホルモ的な時間方向に伸びた入力因子も存在します。脳がこれらから独立して成立する方がむしろ不自然ですね。

自分の意見

人間の依存情報つまり人間のアイデンティティを再構築するのに最低限必要な情報というのは、DNA情報及び人類の組み上げてきた通時的かつ膨大な経験情報にしかないはずです。もし経験情報が偶然違う世界線における人類と、この世界の人類がどちらも同じ人類だという少し強引な仮定をしても良いならば、人類らしさはDNAに含まれるたった750MBほどのヒトゲノムに集約されてしまうわけですがこれは当然間違いです。今の最新のLLMのサイズが数百GBレベルであることを踏まえると、人類はそのアイデンティティを経験情報に依存しきっていることが伺えます。

では経験情報とは何かと言語です。つまり言語だけでもいいのです。知能とは、通時的コミュニ

ケーションができる任意の媒体に現れる特徴なのではないでしょうか。

僕の考えた最強のWeb3

まず現在のインターネットの世界というのは、Web2といわれます。Web3とはその次の世代のもののことです。

- Web1: 情報が一方的にサーバーからクライアントに流れていく
- Web2: サーバーを経由してクライアント同士がインタラクトする（中央集権的）
- Web3: 直接クライアント同士がインタラクトする（分散型）

現在のシステムというのは大抵サーバーがメインとなってクライアントを操るという構成になっています。しかしそれではサーバーにその分負荷がかかりますしエッジデバイスの計算資源を有効活用することが難しいです。またサーバーが何らかの不具合で止まった時に全てのシステムが止まってしまいます。そこで主体をクライアントに分散させることで安定・次世代的なインターネットを作ろうというのがWeb3の思想です。

「僕の考えた最強のWeb3」というのは、互換性を重視したネットワーク構想です。僕はブラウザで動くかどうかがアプリケーションの大きな違いの一つだと考えていますから、このネットワークはブラウザで動くように設計しました。ブラウザでは基本的にHTTP通信しか使うことができないのですが、一つ抜け道がありそれがWebRTCです。

WebRTCとはWeb Real Time Communicationの略でP2P通信というデバイス同士を直接接続するプロトコルの上に構築されたプロトコルです。例えばZoomなどに使われています。

ブラウザ→HTTP→サーバー
ブラウザ→WebRTC→ブラウザ

P2Pはシグナリングという操作を行わないと通信を始めることができません。シグナリングは基本的にはHTTPを用いてサーバーを使って行われますが、デバイス同士の情報を交換すればいいだけなので具体的な方法は規定されていません。

そこでP2PでWeb3を作ろうとしたわけです。しかし一つ大問題が発生します。P2Pを使ってネットワークを作ろうとすると、どうしてもつなげたいサーバー全てに対してシグナリングをして通信を確立しないといけません。しかし大抵のデバイスはP2Pでの同時接続数が技術的な問題で制限されていて大抵数十個単位です。つまりこの方法だと同時に数十個のサーバーにしか通信をできないというわけです。

そこで現行のWebを真似てある工夫をしてみます。何らかのサーバーと接続する際に、直接サーバーと繋ぐのではなく近くのデバイスと接続して伝言ゲームをしていくことでサーバーと接続しようという工夫をします。そうすると同時接続しないといけない数を減らしつつたくさんのサーバーにアクセスできるようになります。

プロトコルについて

まず、基盤の層から話していくと、Web3は革新的なものでありまだまだ実用化には程遠いと言うイメージが(自分にも)あります。なので、自分はできるだけ表層のアプリケーションのみを変更してある意味手軽なWeb3を作りたいと考えています。

そのためにWebRTCという技術を使います。これはブラウザから使えるP2PをするためのAPIで、本来はZoomなどのWeb会議に使われるAPIです。このAPIには、dataChannelというUDPに信頼性を持たせたチャンネルがあり、それを利用して低レイヤーをできるだけ触らずにWeb3を実現します。

イメージは、WebRTCの上にOSI参照モデルでいうIPより上の階層をもう一度積み上げるようなものです。

WebRTC -> DataChannel -> RoutingLayer -> TransportLayer -> ApplicationLayer

このアプローチをWebTCPと名づけました。またWeb3の目玉の一つである分散アプリケーション・分散ストレージ・ブロックチェーンなども基盤として提供することで、分散アプリケーションの開発を簡易にし、Web3の普及を図ります。

ここまでのWeb3のアイデアをWebRTCの上に積み上げようとしている人は、ネットで自分が調べた限りいませんでした。

こんなことを考えていくうちに、これはWebRTCでなくても任意の同様なプロトコル上で積み上げることができると思うので、チャンネルを IRTCChannel として抽象化しブラウザだけでなくクライアントのネイティブアプリ・サーバーでも動くようにしました。

ただし、WebRTC以外の同じような制限されたプロトコル上ではそれ同士の互換性が直接ありませんし、ブラウザ外でわざわざWebRTCをエミュレートするのも大変です。

そこでネットワーク自体をいろんな基盤プロトコル上に存在できるものとして抽象化し、それらの複数種類のネットワークの繋げ役（Web2でいうプロキシサーバーのようなもの）を Adapter という自由度の高いサーバーが担うアプリケーションとしてさらに抽象化します。

そうすると、Web2のネットワークとWeb3のネットワークを互換性を持って接続することが可能となります。

例えば、webtcp://[...].[network_name].global の形でWeb2におけるドメインに接続するように global トップレベルドメインとしてアクセスできるようにしてみます。ブラウザなど制限された環境ではWebRTCで WebTCP2Web2Adapter と通信をし、制限がない場合は直接アクセスするようにします。

global トップレベルドメインを定義することは他にも利点があって、例えばドメイン解決のためのネームサーバーを分散アプリケーション・ブロックチェーンとして管理することができます。例えば domain.global ドメインで統一的にドメイン取得ができれば嬉しいですね。Web3を使うと、アプリ

ケーションを実行する物理デバイスが分散することで、むしろ抽象化されたアプリケーションは集中しても問題なくなるというわけです。

またこの仕組みだと、現時点では特別なソフトウェアが必要なTor技術を理想したダークウェブも良くも悪くも簡単に実装できます。

Webの技術

Web2で使われていたのは主にHTTP(S)通信でこれは Hyper Text Transport Protocol(Secure) の略です。ここでの Hyper Text とはHTMLなどの文書形式のことで、枯れた技術ですがあくまでも情報提供のためのプロトコルであり色々な意味での限界があります。

ここでは簡略化のためWebというプラットフォームの問題をHTTP(S)になすりつけています。有識者の方は適宜HTTP(S)を Web標準 に置き換えてください。HTTP(S)というのはあくまでもTCPの上に載った薄いラッパーです。

「枯れた」というのは長年使われてきたおかげで信頼性がありよく出回っているという良い意味です。Webはもちろん現在も活発で更新され続けていますが「枯れた」ということもできるだろうというのが僕の意見です。

大きな問題点の一つにXMLの重さ・ブラウザで動くプログラミング言語であるJavaScriptの根本的な速度限界・ブラウザの開発複雑化などがあると思います。その解決の一つに僕は、WASIとP2P通信(もしくは同種の次世代プロトコル)の組み合わせを推しています。

まずは現状を理解するところから始めましょう。現状のWebは僕からするとすでに後方互換性のせいで良くも悪くも「継ぎ接ぎ状態」です。

Webは、Html(とcss)とJavaScriptで構成されます。Htmlは実はXMLを継承している言語で、XMLというのは昔使われていたデータトランスポートです。今はほとんどJSONになっていますね。JSONはXMLに比べて軽量でパースしやすいという性質がありこれが代替となった理由です。

データトランスポート(独自定義)：データを通信しあうためのファイル形式のこと

パース：ファイルを読み込み機械が扱えるような状態に変換すること

XML：`<data name="名前?">ほにゃらら</data>` のような形式のこと

JSON：`{ data: {name: "名前?", children: "ほにゃらら"}}` のような形式のこと。

ちなみにGoogle開発の*Protocol Buffers*というより挑戦的な技術があります。これはテキストではなくバイナリで保存することでパース処理をなくして高速化しようという何方かと言えば組み込み開発的なアプローチのデータトランスポートで、スキーマがないと動かない代わりにその分最適化してくれます。現代は静的型がトレンドなのでスキーマ必須でもみんな使いそう...

詳しくは公式ドキュメント(英語)をどうぞ：<https://protobuf.dev>

バイナリ：0と1で表す形式のこと。テキスト形式よりも効率的・軽量の傾向がある。機械が直接理解できる。

スキーマ：データの構造を事前に示しておく時に使われる。英文における英文法のような存在。スキーマ必須の場合、ある程度の不自然さができるがその分セキュリティカル。最近トレンド。

型：データをどのような形でメモリに格納するか形式のこと。「静的型」の場合、柔軟性低く・速く・安全で、「動的型」の場合、柔軟性高く・遅く・危険な傾向がある。「静的型」がトレンド。

JSONを継承したマークアップを作るのもいいと思いますが、マークアップとして使うなら最適かつ互換性もありAIとの相性もいいMD(MarkDown)という形式があります。

マークアップ：ドキュメントを構成するためのファイル形式全般のこと

それはさておき、僕は宣言的なマークアップ言語なんていないと考えていて、手続き的なUI構成WASIを宣言的マークアップ言語からビルド時生成すればいいのです。

UI：ユーザーインターフェースの略。ここではデバイスで見るグラフィカルなインターフェースを指す。

WASI: ブラウザで動くポータブルなバイナリ形式で保存できるプログラムである
WASM(WebAssemblyの略)が外部インターフェースを規格化した結果、ブラウザから飛び出し
WASM Runtimeという種のソフトウェア上で動くプログラムおよびその規格のこと。

ポータブル：この場合、あらゆる環境(パソコン・スマホ)などで動くという意。

外部インターフェース：OS・アーキテクチャ依存の部分の意。

アーキテクチャ：平たくいうとCPUの種類のこと。(本当はソフトウェアレベルで定義されているCPUの種類のこと。)

手続き的・宣言的：僕の書いた別の記事「日常へIT設計という概念を持ち込んでみる」を参照。

ビルド時生成する：開発側のコンピュータで変換し変換結果をデバイスに配信するというアプローチのこと。

またJavaScriptというものは必要ありません。TypeScriptなどのように開発時に型が必要というのは痛いほどわかりますが、ビルド時に型情報を消してしまうのは明らかに非効率的です。時代の流れで消えてしまったasm.jsというものがよっぽど良かったような気がします。

TypeScriptとはJavaScriptの欠点の一つである型がないということを解消するために型情報を書けるようにMicrosoftが開発したJavaScriptの派生言語。ただブラウザ互換性のためTypeScriptは実行前

に型情報を消しJavaScriptへ変換されてから実行される。静的型言語のメリットの一つである実行速度の速さは享受できないが、そもそもJavaScriptはJIT(Just in time)コンパイルによってホストに意味わからないくらい極限まで最適化されて実行されるので、特殊な分野以外では速度の遅さはほとんど気にならない。JavaScriptの高速性に甘えた結果、開発時には型情報が決定しているのに、実行時は型が不確定というどう考えても非効率なシステムになってしまっていると言っている。

この「非効率」というのはv8エンジンのような極端に高性能なJavaScriptエンジンのおかげで消えているように思いますが、ブラウザの開発が極限まで難しくなるのでブラウザ開発が大企業独占となってしまうという側面があります。ブラウザというのはたくさんあるように思うかもしれませんが、現在現役のJavaScriptエンジンは大抵v8エンジン(Chromeの中身)かJavaScriptCore(Safariの中身)かSpiderMonkey(FireFoxの中身)のみで、それぞれGoogle・Apple・Mozillaという大企業によるものです。

ちなみにMicrosoftは昔はJavaScriptエンジンを作っていたんですが、上記の会社に競争で負けて最新のWebに追随することを諦めています。現在のEdgeの中ではv8エンジンが動いています。あの大企業のMicrosoftが諦めた開発ですからJavaScriptエンジンの開発のレベルの高さはよくわかると思います。

そしてそれらとは対照的にwasmランタイムは、静的型付けですし、アセンブリレベルなのでランタイム側で実装しないといけない部分(JITコンパイラなど)は少ないわけではありませんが、JavaScriptよりはマシです。実際企業の垣根を超えたプログラマのコミュニティやベンチャー企業などでも開発されています。

開発の敷居が低いのは、簡単だからという理由の他にもエッジデバイス用のランタイムなどブラウザよりもニッチな分野がたくさんあるからという理由もあります。現在はwasmランタイムは一般ユーザー向けのアプリケーションというよりかは、様々なアプリケーションの基盤として使われている傾向があります。

しかしJavaScriptを急に世の中から消すことは不可能に近いので、JavaScriptがなくてもwasmでアプリが作れるという状態に持っていき段階的にJavaScriptを排除していくというアプローチが現実的です。

このアプローチはすでに大企業により進行しています。JavaScriptなしでwasmを初期化し実行できたり画面を操るDOM APIをwasmから触れるようにしたりscriptタグでwasmを読み込みJavaScriptからESmoduleとして扱えるようにするなどの仕様が現在も慎重に検討されています。